

JPA (suite)

JPA (Suite)

- JPA - Annotations additionnelles
- Hibernate validation
- JPQL

JPA (Annotations)

- `@Entity` suivi de `@Table(name = "nom de table dans la BD")`
- `@Column` précédant un champ (ou variable d'instance)
`@Column(name = "first_name", length=50, nullable=true)`
- `@Transient` - permet d'exclure une propriété d'être mappé dans un champ de la table sous jacente à l'`@Entity`.
- `@Enumerated(EnumType.ORDINAL)` ou `@Enumerated(EnumType.STRING)` pour une propriété qui est d'un type 'Enum'

JPA Annotations

Contrainte d'unicité

```
@Table(  
    uniqueConstraints=  
        @UniqueConstraint(columnNames={"author_id", "number"})  
)  
@Entity  
class Book extends Model {  
    @ManyToOne  
    @JoinColumn(name = "author_id")  
    User author;  
    int number;  
}
```

JPA Transaction

- EntityTransaction tx = em.getTransaction()
- On peut par la suite effectuer du code du genre pour simuler un 'auto-commit'. À noter qu'on a accès à l'id de l'entité après avoir effectué le commit.

```
// Creates and persists an author  
tx.begin();  
author = service.createAuthor(author);  
tx.commit();  
Long id = author.getId();  
  
System.out.println("Author Persisted : " + author);
```

Hibernate validation

- Hibernate Validator

```
<dependency>
  <groupId>javax.validation</groupId>
  <artifactId>validation-api</artifactId>
  <version>2.0.1.Final</version>
</dependency>
```

```
<dependency>
  <groupId>org.hibernate.validator</groupId>
  <artifactId>hibernate-validator</artifactId>
  <version>6.0.13.Final</version>
</dependency>
```

Hibernate validation

```
import javax.validation.constraints.AssertTrue;
import javax.validation.constraints.Max;
import javax.validation.constraints.Min;
import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;
import javax.validation.constraints.Email;

public class User {

    @NotNull(message = "Name cannot be null")
    private String name;

    @AssertTrue
    private boolean working;

    @Size(min = 10, max = 200, message
        = "About Me must be between 10 and 200 characters")
    private String aboutMe;

    @Min(value = 18, message = "Age should not be less than 18")
    @Max(value = 150, message = "Age should not be greater than 150")
    private int age;

    @Email(message = "Email should be valid")
    private String email;
```

Hibernate validation

- @NotEmpty
- @NotBlank
- @Positive, @PositiveOrZero
- @Negative, @NegativeOrZero
- @Past, @PastOrPresent
- @Future, @FutureOrPresent

Hibernate Validation

- `List<@NotBlank String> preferences`
- `public Optional<@Past LocalDate> getDateOfBirth()`

Hibernate Validation

```
User user = new User();  
user.setWorking(true);  
user.setAboutMe("Its all about me!");  
user.setAge(50);
```

```
Set<ConstraintViolation<User>> violations = validator.validate(user);
```

```
for (ConstraintViolation<User> violation : violations) {  
    log.error(violation.getMessage());  
}
```

JPA quelques problèmes

- Lazy initialization des associations
- $N + 1$

JPQL

SELECT
FROM

<select clause>
<from clause>

JPQL

```
SELECT      <select clause>  
FROM        <from clause>  
[WHERE      <where clause>]  
[ORDER BY   <order by clause>]  
[GROUP BY   <group by clause>]  
[HAVING     <having clause>]
```

```
<function>   AVG, COUNT, MAX, MIN, SUM
```

```
<operators>  =, >, >=, <, <=, <>, [NOT] BETWEEN, [NOT] IN,  
              [NOT] LIKE, IS [NOT] NULL, IS [NOT] EMPTY,  
              [NOT] MEMBER [OF]
```

JPQL

```
SELECT      <select clause>  
FROM        <from clause>  
[WHERE      <where clause>]  
[ORDER BY  <order by clause>]  
[GROUP BY <group by clause>]  
[HAVING    <having clause>]
```

```
<num exp.>    ABS, SQRT, MOD, SIZE, INDEX
```

```
<string exp.> CONCAT, SUBSTRING, TRIM, LOWER, UPPER,  
               LENGTH, LOCATE
```

```
<date exp.>   CURRENT_DATE, CURRENT_TIME, CURRENT_TIMESTAMP
```

JPQL

```
SELECT      b  
FROM        Book b  
WHERE       b.unitCost > 100
```

- L'alias 'b' représente une instance de la classe 'Book'
- Dans le 'SELECT', le b représente 0 à plusieurs instances répondant aux critères de la requête
- la clause 'WHERE' montre comment naviguer à partir de l'instance 'b' pour restreindre la requête JPQL.
- 'unitCost' est une variable d'instance de Book.

JPQL - Select

```
SELECT      b  
FROM      Book b
```

```
SELECT      b.title, b.unitCost, b.isbn  
FROM      Book b
```

```
SELECT      AVG(b.unitCost)  
FROM      Book b
```

```
SELECT      COUNT(b)  
FROM      Book b
```

```
SELECT      b.publisher  
FROM      Book b
```

```
SELECT      b.publisher.name  
FROM      Book b
```

```
SELECT      DISTINCT(b.publisher.name)  
FROM      Book b
```

JPQL - WHERE

```
SELECT  
FROM  
WHERE  
b  
Book b  
b.unitCost > 29
```

```
SELECT  
FROM  
WHERE  
b  
Book b  
b.unitCost > 29 AND b.nbOfPage < 100
```

```
SELECT  
FROM  
WHERE  
b  
Book b  
b.unitCost > 29 AND b.nbOfPage BETWEEN 50 AND 90
```

```
SELECT  
FROM  
WHERE  
b  
Book b  
b.title LIKE '%java%'
```

```
SELECT  
FROM  
WHERE  
b  
Book b  
LOWER(b.title) LIKE '%java%'
```

Queries

<<Interface>>

EntityManager

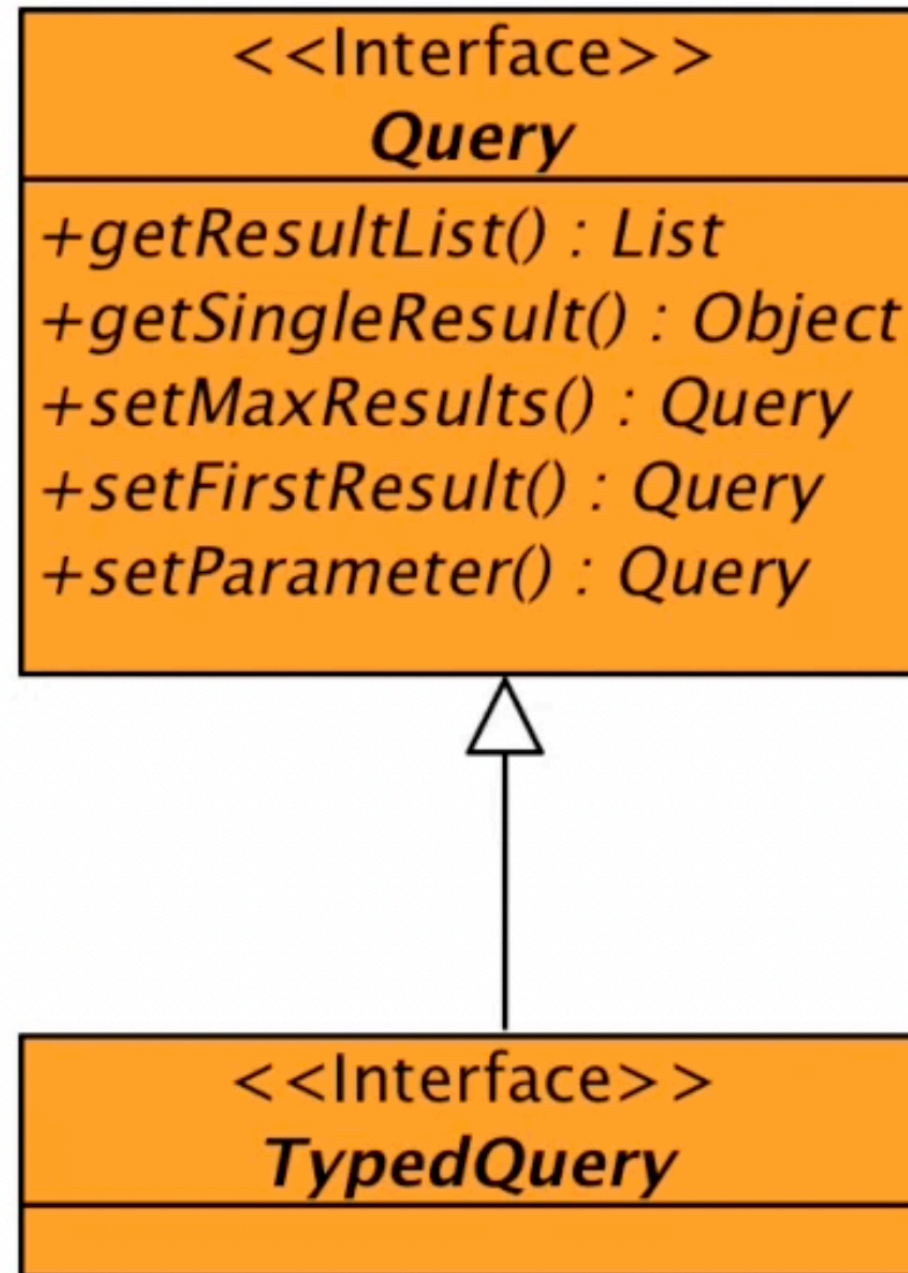
+createQuery(statement : String) : Query

+createQuery(statement : String, type : Class) : TypedQuery

+createNamedQuery(name : String) : Query

+createNamedQuery(name : String, type : Class) : TypedQuery

Queries



Queries

```
Query query = em.createQuery(  
    "SELECT b FROM Book b  
    WHERE b.unitCost > 29 AND b.nbOfPage < 700");
```

```
Book book = (Book) query.getSingleResult();
```

```
TypedQuery<Book> query = em.createQuery(  
    "SELECT b FROM Book b  
    WHERE b.unitCost > 29 AND b.nbOfPage < 700",  
    Book.class);
```

```
List<Book> books = query.getResultList();
```

Queries

```
TypedQuery<Book> query = em.createQuery(
    "SELECT b FROM Book b
    WHERE b.unitCost > ?1 AND b.nbOfPage < ?2",
    Book.class);

query.setParameter(1, unitCost);
query.setParameter(2, nbOfPage);
List<Book> books = query.getResultList();
```

```
TypedQuery<Book> query = em.createQuery(
    "SELECT b FROM Book b
    WHERE b.unitCost > :cost AND b.nbOfPage < :pages",
    Book.class);

query.setParameter("cost", unitCost);
query.setParameter("pages", nbOfPage);
List<Book> books = query.getResultList();
```

Queries

```
TypedQuery<Book> query = em.createQuery(  
    "SELECT b FROM Book b  
    WHERE b.publicationDate < :pubDate",  
    Book.class);  
  
query.setParameter("pubDate", LocalDate.now());  
  
List<Book> books = query.getResultList();
```

Queries

```
Query query = entityManager.createQuery("From Foo");  
int pageNumber = 1;  
int pageSize = 10;  
query.setFirstResult((pageNumber-1) * pageSize);  
query.setMaxResults(pageSize);  
List <Foo> fooList = query.getResultList();
```

Queries

```
String statement = "SELECT b FROM Book b  
                   WHERE b.unitCost > :cost ";  
  
if (hasPages)  
    statement += "AND b.nbOfPage < :pages ";  
if (hasDate)  
    statement += "OR b.publicationDate < :pubDate";  
  
TypedQuery<Book> query = em.createQuery(statement, Book.class);  
query.setParameter("cost", unitCost);  
query.setParameter("pages", nbOfPage);  
query.setParameter("pubDate", publicationDate);
```